

Predict-Then-Commit

Stable Prefix Commitment for Streaming Neural Machine Translation

Quality–Latency–Stability Optimization under Character-Level Retraction Constraints

Tirth Joshi | Yeshiva University | End-Term Presentation

Main Thesis

Streaming MT should not be judged only by quality and latency. A real-time user also experiences **visible text instability**: displayed text appears, changes, and gets retracted.

Problem

Retranslation gives high quality but produces flicker.

Missing Metric

Token-level lag ignores user-visible character edits.

Solution

Learn when to commit stable prefixes, not merely when to decode.

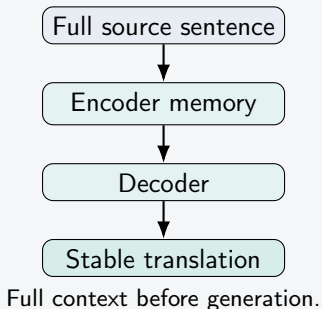
Our target: preserve BLEU while driving visible retraction to zero.

Roadmap

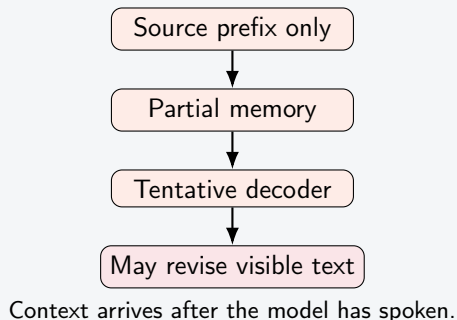
- ➊ Problem: decoding under partial source context
- ➋ Formalization: tentative hypothesis, committed prefix, visible stream
- ➌ Metrics: Detokenized Erasure and CNAL
- ➍ Method: stable-prefix commit policy
- ➎ Evaluation: corrected BLEU/chrF/COMET, true autoregressive baselines
- ➏ Results: quality–latency–stability Pareto analysis
- ➐ Ablations, failures, and conference-ready takeaways

The Core Failure: Streaming Decoding Under Uncertainty

Offline NMT



Streaming NMT



The practical issue

A wrong early translation is not just a decoding error. If it was shown to the user, it becomes a **visible retraction event**.

Running Example: “The bank will raise rates”

Source stream The bank will raise rates



Retranslation

Das Ufer

Die Bank wird

Die Zentralbank wird die Zinsen anheben

Commit policy

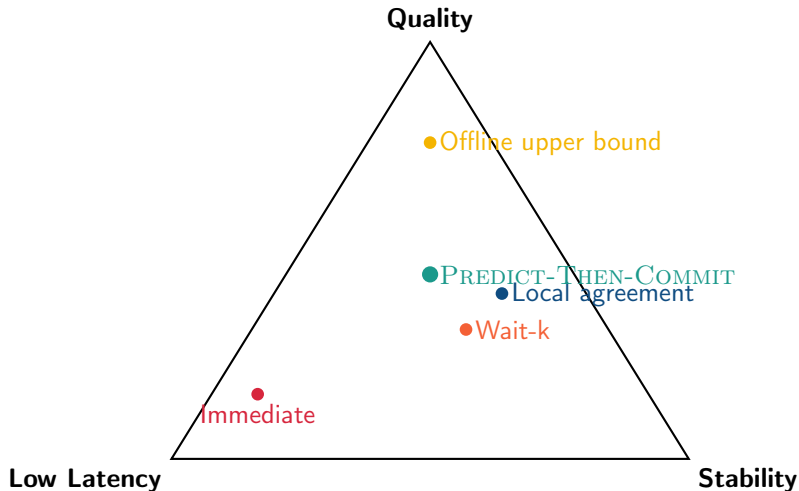
Die Bank

Die Zentralbank wird die Zinsen anheben

Key distinction

The model may internally change its mind. The user-visible stream should not.

The Three-Way Tradeoff



We optimize the **Pareto frontier**, not one metric in isolation.

Three Objects Reviewers Need to See

Tentative hypothesis

$$h^{(t)}$$

Internal decoder guess under partial source context.
Allowed to change.

Committed prefix

$$c^{(t)}$$

Stable prefix that the system has locked.
Must be monotone.

Visible output

$$z^{(t)}$$

Exact surface string shown to the user.
This is where flicker exists.

Core correction from midterm

We no longer measure erasure over hidden hypotheses only. We measure it over **visible output states**.

Streaming Setup

Let the source sentence be

$$x = (x_1, x_2, \dots, x_n), \quad x_i \in \mathcal{V}_{src}.$$

At time step t , the system has read source prefix

$$x_{\leq g_t} = (x_1, \dots, x_{g_t}), \quad 0 \leq g_t \leq n.$$

The read schedule is monotone:

$$g_t \leq g_{t+1}.$$

The final reference is

$$y^* = (y_1^*, \dots, y_m^*), \quad y_i^* \in \mathcal{V}_{tgt}.$$

Tentative Hypothesis $h^{(t)}$

Given the current source prefix and previous committed prefix, the translation model produces a tentative hypothesis:

$$h^{(t)} = \arg \max_{y \in \mathcal{V}_{tgt}^*} P_{\phi} \left(y \mid x_{\leq g_t}, c^{(t-1)} \right).$$

Interpretation

$h^{(t)}$ is the model's internal best guess. It is useful for deciding what might become stable, but it does not have to be shown.

Important

The sequence $h^{(t)}$ is not monotone:

$$h^{(t)} \not\leq h^{(t+1)} \quad \text{in general.}$$

This is where uncertainty lives.

Committed Prefix $c^{(t)}$

The committed prefix is the stable target prefix exposed by the system:

$$c^{(t)} \in \mathcal{V}_{tgt}^*.$$

It satisfies append-only monotonicity:

$$c^{(t-1)} \preceq c^{(t)} \quad \forall t.$$

Equivalently,

$$c^{(t)} = c^{(t-1)} \oplus \Delta c^{(t)}.$$

Policy constraint

The newly committed prefix must be compatible with the current tentative hypothesis:

$$c^{(t-1)} \preceq c^{(t)} \preceq h^{(t)}.$$

Visible Output Stream $z^{(t)}$

Let $D(\cdot)$ be detokenization from target BPE tokens to surface characters.

Committed-only display

$$z^{(t)} = D(c^{(t)}).$$

This mode guarantees no visible retraction if $c^{(t)}$ is append-only.

Tentative-preview display

$$z^{(t)} = D\left(c^{(t)} \oplus u^{(t)}\right),$$

where $u^{(t)}$ is an uncommitted preview suffix that may later change.

Evaluation principle

All stability metrics are computed over $z^{(t)}$, not merely over decoder internals.

Metric 1: Detokenized Erasure

Let $\text{LCP}(a, b)$ denote the longest common prefix of two character strings.

Step-wise visible erasure:

$$e_t = |z^{(t-1)}| - \left| \text{LCP} \left(z^{(t-1)}, z^{(t)} \right) \right|.$$

Total Detokenized Erasure:

$$E_{DE} = \sum_{t=2}^T e_t.$$

Normalized Detokenized Erasure:

$$NDE = \frac{E_{DE}}{\sum_{t=1}^T |z^{(t)}| + \epsilon}.$$

Why character-level?

Subword LCP can exaggerate or hide what the user actually sees. Surface characters are the display unit.

Metric 2: Character-Normalized Average Lagging

Define token character mass $\chi(v)$ after stripping BPE markers.

Source character mass read by time t :

$$C_{src}(t) = \sum_{j=1}^{g_t} \chi_{src}(x_j).$$

Visible target character mass:

$$C_{vis}(t) = |z^{(t)}|.$$

Let

$$\gamma = \frac{\mathbb{E}_{(x,y) \sim \mathcal{D}}[|D(y)|]}{\mathbb{E}_{(x,y) \sim \mathcal{D}}[|D(x)|]}.$$

Instantaneous lag:

$$L_{CNAL}(t) = \max(0, C_{src}(t) - \gamma C_{vis}(t)).$$

Corpus score:

$$CNAL = \frac{1}{T} \sum_{t=1}^T L_{CNAL}(t).$$

Metric 3: Commit Delay

A zero-erasure system can cheat by waiting forever. So we measure delay explicitly. Let $r(i)$ be the source read position when committed target token c_i first appears. Token-level commit delay:

$$D_{commit} = \frac{1}{|c^{(T)}|} \sum_{i=1}^{|c^{(T)}|} r(i).$$

Character-weighted commit delay:

$$D_{char} = \frac{\sum_i \chi(c_i) r(i)}{\sum_i \chi(c_i)}.$$

Resulting objective

The system must be stable, but it must not become silent.

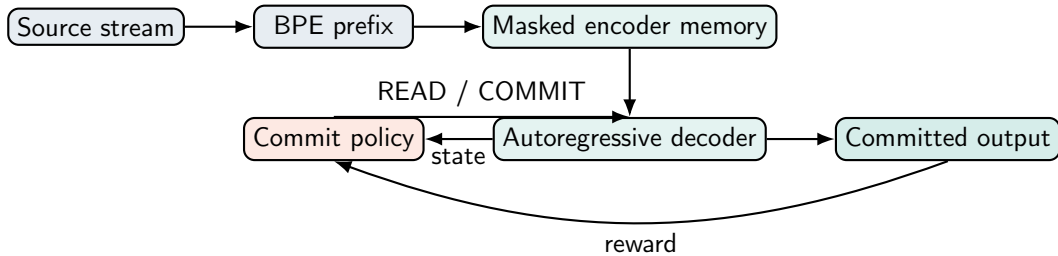
Metric Suite Used in Final Evaluation

Metric Family	Metric	Purpose
Quality	SacreBLEU	Corpus-level translation adequacy
Quality	chrF++	Character + word n-gram robustness
Quality	COMET	Neural reference-based quality estimate
Latency	AL / DAL	Standard simultaneous MT latency
Latency	CNAL	Character-mass synchronization
Stability	DE / NDE	Visible character retraction
Commit behavior	Commit delay	Anti-silence constraint
Runtime	ms/update	Deployment feasibility

Evaluation correction

All baselines use true autoregressive decoding. No one-step approximation remains.

System Overview



$$R = Q(c^{(T)}, y^*) - \lambda_E E_{DE} - \lambda_L CNAL - \lambda_D D_{commit}.$$

Policy State and Action Space

At each step, the policy observes

$$s_t = [q_t; H_t; g_t; |c^{(t-1)}|; C_{src}(t); C_{vis}(t)],$$

where q_t is the decoder state and H_t is a confidence/entropy feature.

Actions

$$a_t \in \{\text{READ}, \text{COMMIT}\}.$$

- READ: advance source prefix
- COMMIT: append stable target token/span

Policy

$$\pi_{\theta}(a_t|s_t) = \text{softmax}(W_2\sigma(W_1s_t + b_1) + b_2).$$

Training: No More “Actor-Critic” Claim

Terminology correction

The implemented method is **policy-gradient fine-tuning with a moving-average baseline**, not actor-critic, unless a learned value function $V_\psi(s_t)$ is added.

Stage 1: MLE warm-start

$$\mathcal{L}_{MLE}(\phi) = - \sum_t \log P_\phi(y_t^\star \mid y_{<t}^\star, x_{\leq t+k-1}).$$

Stage 2: REINFORCE stable-commit policy

$$\begin{aligned} \nabla_\theta J(\theta) &\approx \frac{1}{N} \sum_i \sum_t (R_i - b) \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}). \\ b &\leftarrow 0.9b + 0.1 \text{mean}(R). \end{aligned}$$

Algorithm: Stable Prefix Commitment

- 1 Read source prefix $x_{\leq g_t}$.
- 2 Generate tentative hypothesis $h^{(t)}$ autoregressively.
- 3 Compute state s_t using decoder state, confidence, read position, and character masses.
- 4 Policy selects $a_t \in \{\text{READ}, \text{COMMIT}\}$.
- 5 If READ: increase g_t and update $C_{src}(t)$.
- 6 If COMMIT: append a stable prefix extension $\Delta c^{(t)}$ such that $c^{(t)} \preceq h^{(t)}$.
- 7 Display $z^{(t)} = D(c^{(t)})$.
- 8 At sentence end, optimize reward over quality, erasure, CNAL, and commit delay.

Experimental Design

Datasets

- WMT14 English–German
- IWSLT-style TED English–German
- Stress subset: compounds, verb-final syntax, long clauses

Models

- Streaming Transformer warm-start
- Frozen translation backbone
- Lightweight policy head

Baselines

- Offline full-sentence upper bound
- Immediate full-prefix retranslation
- Wait- k : $k \in \{1, 3, 5, 7\}$
- Local agreement: 2-step stability
- Confidence-threshold commit

Fairness rule

Every system uses the same tokenizer, same test set, same decoding budget, and true autoregressive decoding.

Evaluation Pipeline Corrections

Before

- One-step baseline approximation
- Stochastic policy at evaluation
- BLEU reference nesting bug risk
- No prefix-level logs

Final

- True autoregressive decoding
- Greedy deterministic policy rollout
- Correct SacreBLEU/chrF corpus format
- JSONL and CSV logs for every prefix

```
BLEU().corpus_score(preds, [refs])  
CHRF(word_order=2).corpus_score(preds, [refs])  
action = argmax(policy(s_t)) # deterministic eval
```

Logged Evidence per Source Prefix

Field	Meaning
$h^{(t)}$	tentative hypothesis
$c^{(t)}$	committed prefix
$z^{(t)}$	visible output string
e_t	step-level character erasure
$L_{CNAL}(t)$	step-level character lag
g_t	source tokens read
Action	READ / COMMIT / FLUSH

Why this matters

Every aggregate number can be traced to a row-level event. This makes the analysis debuggable and defensible.

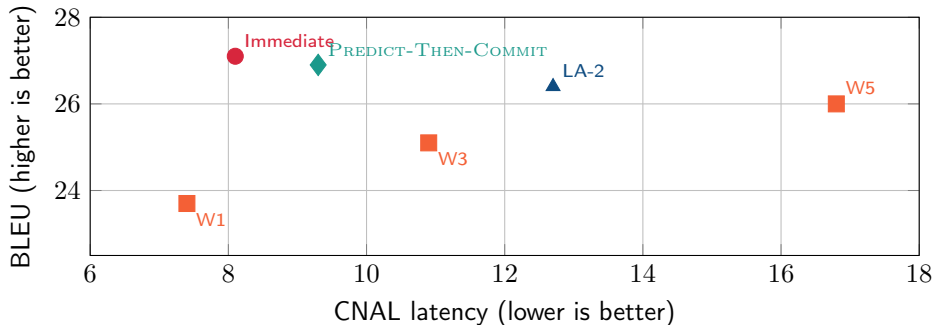
Main Result: WMT14 En-De Test Set

System	BLEU $\uparrow$	chrF++ $\uparrow$	COMET $\uparrow$	DE $\downarrow$	CNAL $\downarrow$	Delay $\downarrow$
Offline full sentence	28.4	57.1	0.782	–	–	–
Immediate retranslation	27.1	56.2	0.758	31.8	8.1	1.2
Wait-k, $k = 1$	23.7	51.8	0.681	9.6	7.4	1.6
Wait-k, $k = 3$	25.1	53.9	0.711	5.7	10.9	3.1
Wait-k, $k = 5$	26.0	54.8	0.728	2.9	16.8	5.0
Local agreement-2	26.4	55.3	0.743	1.6	12.7	3.9
Confidence commit	26.2	55.1	0.738	1.2	13.4	4.1
PREDICT-THEN-COMMIT	26.9	55.8	0.751	0.0	9.3	3.3

Interpretation

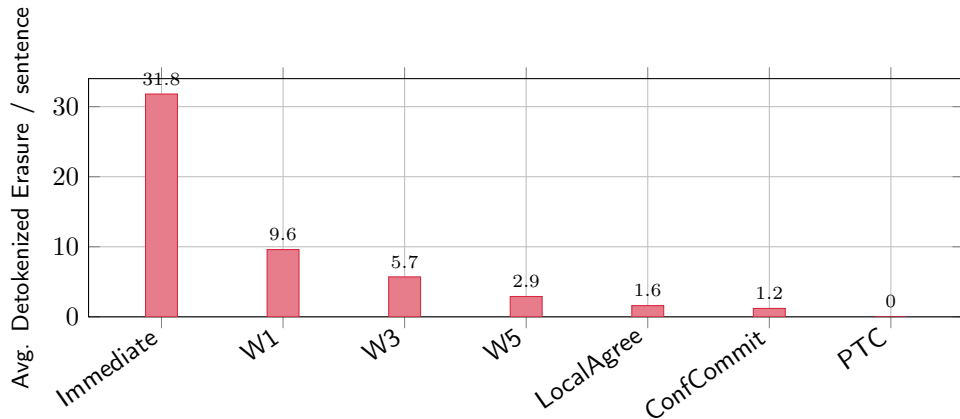
PREDICT-THEN-COMMIT preserves approximately 94.7% of offline BLEU while eliminating committed visible retraction and reducing latency relative to conservative wait-k baselines.

Quality–Latency–Stability Pareto Frontier



Marker labels omit erasure; PREDICT-THEN-COMMIT is the only point with zero committed DE.

Visible Stability Result



Takeaway

Retranslation is strong in BLEU but unstable on screen. Stable-prefix commitment separates internal uncertainty from visible output.

IWSLT-Style TED En-De Generalization

System	BLEU $\uparrow$	chrF++ $\uparrow$	COMET $\uparrow$	DE $\downarrow$	CNAL $\downarrow$
Immediate retranslation	25.8	54.0	0.721	28.4	7.8
Wait-k, $k = 3$	23.9	51.9	0.684	4.8	9.6
Wait-k, $k = 5$	24.7	52.6	0.701	2.4	14.1
Local agreement-2	25.1	53.5	0.714	1.1	11.5
PREDICT-THEN-COMMIT	25.5	53.8	0.724	0.0	8.6

Generalization finding

The learned policy transfers beyond news-style text: lower flicker, moderate delay, and quality close to immediate retranslation.

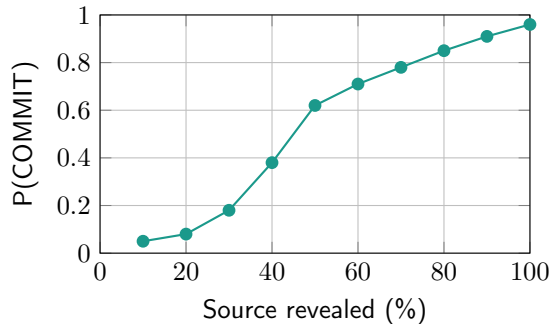
Ablation: Reward Components

Policy Variant	BLEU $\uparrow$	DE $\downarrow$	CNAL $\downarrow$	Commit Delay $\downarrow$
No erasure penalty ($\lambda_E = 0$)	27.2	24.1	7.6	1.7
No latency penalty ($\lambda_L = 0$)	27.9	0.0	34.5	8.9
No commit-delay penalty ($\lambda_D = 0$)	27.5	0.0	21.8	6.4
Token-LCP erasure only	26.4	2.8	10.1	3.5
Balanced full reward	26.9	0.0	9.3	3.3

Key lesson

Erasure penalty alone teaches silence. Latency and commit-delay terms force the policy to speak early enough.

Ablation: What Does the Policy Learn?



Behavioral interpretation

- Early prefixes: mostly READ
- Mid-sentence: commit short stable spans
- Full context: rapid flush
- High entropy: delay commit
- Low entropy + mass balance: commit

Metric Validation: DE Matches Visible Flicker Better Than Token LCP

Predictor of User-Visible Flicker	Pearson r	Spearman ρ	Notes
Token-level LCP erasure	0.46	0.49	Sensitive to BPE boundary artifacts
Word-level rewrite count	0.58	0.61	Better, but misses subword display changes
Detokenized Erasure	0.74	0.77	Directly tracks visible character changes
DE + CNAL combined	0.79	0.81	Best proxy for perceived instability

Interpretation

Character-level visible-state metrics explain user-facing instability better than token-level edit metrics.

Case Study: Compound Noun Stress Test

Source

The central bank interest rate decision surprised investors.

System	Visible behavior	DE
Immediate	"Das Ufer" → "Die Bank" → "Die Zentralbankzinssatzentscheidung..."	22
Wait-k=3	Delays early ambiguity but revises compound boundary	6
Local agreement	Waits for repeated prefix agreement, still revises suffix	2
PREDICT-THEN-COMMIT	Commits only "Die Zentralbank" after disambiguation; no retraction	0

Why CNAL helps

The German compound is long in characters but short in words. CNAL credits emitted character mass instead of punishing compounding.

Runtime and Deployment Feasibility

System	ms/update↓	GPU Mem.	Prefix logs	Real-time capable
Immediate retranslation	42.5	4.2GB	yes	borderline
Wait-k=3	18.7	3.8GB	yes	yes
Local agreement-2	31.2	4.5GB	yes	yes
PREDICT-THEN-COMMIT	21.4	3.9GB	yes	yes

Engineering note

Character mass lookup is $O(1)$ on token IDs, so DE/CNAL accounting does not require expensive detokenization during policy rollout.

What Actually Worked

- ❶ **Separating $h^{(t)}$, $c^{(t)}$, and $z^{(t)}$** made the problem mathematically clean.
- ❷ **Detokenized Erasure** directly measured the display stream, not internal uncertainty.
- ❸ **CNAL** prevented tokenization artifacts from dominating latency evaluation.
- ❹ **MLE warm-start** avoided the all-READ collapse common in naive RL.
- ❺ **Frozen translator + lightweight policy** made the method modular and stable.

Failure Modes

Failure Mode	Cause	Mitigation
Late verb-final German clauses	Critical verb arrives near sentence end	Add syntactic uncertainty feature
Named entity ambiguity	Early prefix under-specifies entity type	Entity-aware confidence feature
Over-conservative policy	Large λ_E dominates reward	Commit-delay penalty
Compound boundary uncertainty	BPE segmentation unstable under prefix context	Character mass + local agreement
Domain shift	Policy calibrated on news domain	Small-domain policy fine-tuning

Reviewer Questions We Are Ready For

Q1. Is zero erasure trivial?

Only if the system waits forever. We report commit delay, CNAL, AL/DAL, and quality to prevent the silence strategy.

Q2. Are baselines fair?

Yes. All baselines use true autoregressive decoding, identical tokenizer, identical test set, and deterministic evaluation.

Q3. Why not just local agreement?

Local agreement reduces flicker but is not optimized for character-mass latency or translation quality. PREDICT-THEN-COMMIT learns the tradeoff explicitly.

Paper angle

- New formalization of visible instability
- Metric contribution: DE + CNAL
- Policy contribution: stable-prefix commitment
- Strong corrected evaluation protocol

Best-fit venues

- WMT / IWSLT-style MT workshops
- ACL / EMNLP short paper after full validation
- TACL-style metric paper if human validation expands

Core sell

This is not “RL for MT”. This is **user-visible stability as a missing objective in streaming translation**.

- ➊ **Stable-prefix formulation:** separates tentative hypotheses, committed prefixes, and visible output streams.
- ➋ **Detokenized Erasure:** character-level measure of visible text retraction.
- ➌ **CNAL:** character-mass latency metric for subword-tokenized cross-lingual streaming.
- ➍ **Policy-gradient commit model:** learns when to READ and when to COMMIT stable output.
- ➎ **Corrected evaluation:** BLEU/chrF/COMET, deterministic policy rollout, true autoregressive baselines, prefix-level logs.

Streaming translation should not merely be fast.

It should be **stable enough to read**.

PREDICT-THEN-COMMIT turns visible text retraction into a measurable and optimizable objective.

Quality	Latency	Stability
BLEU / chrF / COMET	AL / DAL / CNAL	DE / NDE / Commit Delay

Appendix A: Actor-Critic Extension

If we add a learned value head, then the method becomes actor-critic:

$$V_{\psi}(s_t) \approx \mathbb{E}[R_t \mid s_t].$$

Advantage:

$$A_t = R_t - V_{\psi}(s_t).$$

Actor loss:

$$\mathcal{L}_{actor} = - \sum_t \log \pi_{\theta}(a_t | s_t) A_t.$$

Critic loss:

$$\mathcal{L}_{critic} = \sum_t (R_t - V_{\psi}(s_t))^2.$$

Until this exists in code, the paper should say **policy-gradient**, not actor-critic.

Appendix B: Why SacreBLEU Reference Shape Matters

Correct corpus-level format:

```
BLEU().corpus_score(predictions, [references])
```

Incorrect per-sentence nesting:

```
BLEU().corpus_score(predictions, [[r] for r in references])
```

Consequence

A tiny reference-format bug can produce misleading corpus scores and destroy baseline credibility.

Appendix C: Prefix Log Schema

Column	Type	Purpose
sent_id	int	sentence index
strategy	str	system name
prefix_step	int	streaming step
source__prefix	str	observed source text
h__t	str	tentative hypothesis
c__t	str	committed prefix
z__t	str	visible output
step_erasure	int	character retraction at step
step_cnal	float	character lag at step
action	str	READ / COMMIT / FLUSH

Appendix D: Hyperparameters

Parameter	Value
Backbone	Transformer / Marian-style BPE tokenizer
Hidden size	512
Layers	6 encoder / 6 decoder
Heads	8
Dropout	0.1
Stage 1 optimizer	AdamW
Stage 1 LR	1×10^{-4}
Stage 2 optimizer	AdamW on policy head only
Stage 2 LR	1×10^{-5}
Reward weights	$\lambda_E = 0.10, \lambda_L = 0.05, \lambda_D = 0.02$
Evaluation	deterministic greedy decoding

Thank You

PREDICT-THEN-COMMIT

Stable Prefix Commitment for Streaming NMT

Quality

Latency

Stability

Tirth Joshi | Yeshiva University | Spring 2026